

Technical progress toward Phase II cable routing

GG boy Team | EBiM Task 1: Cable Routing & Plugging | 10 September 2026

Abstract

We report a reproducible development pipeline for moving from synchronized dual-arm teleoperation data toward real-robot cable routing. The contribution is a verified data and runtime foundation, leakage-controlled exploratory fitting, portable inference, and explicit deployment contracts. It is not a deployable autonomous policy. No robot action was sent, and no simulation or real-robot task-success rate is claimed.

The private recording contains 19,216 synchronized frames at 10 Hz; 13,186 are numerically complete. A prior read-only acceptance pass decoded 76,864 RGB/depth images and verified all 76,927 transferred files. We construct 474 training and 112 development-validation windows with temporal embargoes and provisional annotation-boundary exclusions. Three small diagnostic fits are evaluated against a hold-current-arm baseline.

The absolute-target MLP, arm-residual MLP, and a fixed RGB/state ridge diagnostic achieve validation arm RMSE of 0.100292, 0.067558, and 0.066940 rad respectively, versus 0.033563 rad for holding measured joints. These negative results show that model fitting alone does not establish useful control. The report documents the resulting limitations and concrete integration work rather than presenting a reconstruction metric as cable-routing success.

Submission scope

This report is prepared for technical-progress review. The official submissions README describes a Technical Report alternative for teams without a deployable policy [1]. The currently located Issue form is the runnable-policy form; the technical-report form and its Phase II routing require organizer confirmation. Team registration and any team-specific deadline must be checked before publication. No official submission has been made by this work.

Contributions

(1) Read-only data verification and documented action/observation semantics. (2) Guarded time-block splitting with training-only normalization. (3) Reproducible negative-result baselines and error stratification. (4) Portable offline inference and a streaming observation contract that rejects malformed or discontinuous input. (5) A locally verifiable, source-only review package that excludes private artifacts.

Data, provenance and validation protocol

Recording and interpretation

The dataset is a single continuous overlapping recording, not a collection of independent successful episodes. Observations contain left FR3 joints 1-7, right FR3 joints 1-7, and left/right gripper knuckle positions, all interpreted in radians under the recorded field contract. Targets contain the corresponding 14 raw GELLO arm positions and two recorded gripper command values. Knuckle radians and gripper command units are not interchangeable. Raw GELLO targets have not been verified as directly executable commands.

RGB is JPEG; depth is lossless uint16 PNG. The numeric validity mask denotes readable synchronized fields, not expert quality or task completion. Depth metric scale, cross-camera/task extrinsics and independent host-clock calibration remain unverified. Pedal classes are retained as recorded; the B-button translation-versus-turn mapping is unresolved. No missing values or success labels are fabricated.

Provisional annotations

There are 21 coarse intervals and 615 candidate windows in the annotation overlay. All expert-training approvals are false and task outcomes are unknown. Operator-reported coverage includes cable acquisition, successive O fixtures, C-fixture manipulation, mistakes and recovery. Y-checkpoint and final plugging coverage are unconfirmed. Coarse labels and boundary uncertainty support exclusion and stratification only; they do not establish successful demonstrations.

Development split

At 10 Hz, fixed 600-frame blocks follow a deterministic three-training/one-validation pattern. Both edges of each block have a 100-frame embargo. Windows contain 20 frames with stride 10 and must remain within one numerically valid provisional interval. Recorded annotation-boundary uncertainty is additionally excluded. The retained sets have 474 training and 112 validation windows; the closest cross-set frames are 201 frames, or 20.1 seconds, apart.

State normalization for the MLP is calculated from 4,960 unique retained training frames; target statistics use training endpoints. The RGB ridge normalizes features using training windows only. Overlapping windows within each partition remain correlated. Since the first validation result informed the residual-model revision, this is a development-validation set, not an independent final test. A temporal embargo does not demonstrate cross-scene generalization.

Models and reproducibility

State-history MLP diagnostics

A causal 20 x 16 measured-state history is flattened to 320 values and passed through a 128-unit ReLU hidden layer and a 16-output linear layer. Adam uses learning rate 0.001, batch size up to 128, gradient-norm clipping at 1.0 and seed 42. Each run is fixed at 300 updates. The absolute model reconstructs the contemporaneous raw operator target. The residual variant predicts arm target minus current measured arm position; gripper outputs remain in recorded command units.

The two runs used the existing host PyTorch 2.5.1 / CUDA 12.4 environment on an RTX 3060 with 12 GB VRAM. The training/evaluation loops took 2.13 and 2.05 seconds; environment startup and data loading are additional. Deterministic Torch algorithms and the documented CUBLAS workspace setting were used. This was a smoke test, not a converged policy training campaign.

RGB/state ridge diagnostic

The third experiment uses both wrist RGB images at the window endpoint, resized with fixed BOX averaging to 8 x 6 RGB cells per camera. These 288 color features are concatenated with the last 16-state observation and the 16-dimensional displacement across the history, for 320 total features. A closed-form ridge model uses a fixed regularization coefficient of 10, selected before evaluating its validation targets. It predicts arm residuals and raw gripper targets. There is no semantic fixture detector, pretrained visual backbone, parameter sweep or task-success supervision.

The RGB diagnostic ran on CPU in 12.64 seconds including image decoding, fit and export verification. Its compact visual features constitute a diagnostic for image/state correlation, not verified cable geometry or grasp perception.

Saved evidence

Each run saves configuration, split indices, normalized statistics or equivalent feature statistics, weights, predictions, evaluation and logs. MLP runs additionally save PyTorch checkpoints and environment details. Exported NumPy MLP predictions match saved GPU validation predictions within 4.77e-7 and 2.39e-7 maximum absolute difference. The visual model is independently reloaded and checked against its batch predictions. Source and result archives have SHA256 manifests; private data and model artifacts are kept outside the review source ZIP.

Results and error analysis

Method	Train arm RMSE	Val. arm RMSE	Val. gripper RMSE
Training-target mean	0.322464	0.417457	0.367555
Hold current arms	-	0.033563	Not comparable
Absolute MLP	0.024563	0.100292	0.071090
Residual-arm MLP	0.011885	0.067558	0.088204
RGB/state ridge	0.011717	0.066940	0.069148

Arm errors are radians; gripper errors are raw recorded command values. An aggregate combining the two units is not reported. Hold-current applies only to measured arm joints; directly substituting knuckle angles for gripper commands would be invalid. Lower error here is target reconstruction, not manipulation success.

What the negative result shows

Both MLPs fit training endpoints substantially better than development validation. The RGB diagnostic reduces gripper error compared with the two MLP runs, but still has roughly twice the arm error of hold-current. A training-mean arm-residual correction gives 0.032416 rad validation RMSE in a post-hoc diagnostic; this modest reconstruction difference is not evidence of useful autonomous action.

Approximately 79.37% of the residual MLP validation arm squared error comes from the right-arm seventh joint. Its error also exceeds hold-current in the approach, recovery-approach, regrasp and C-routing candidate strata. For higher-motion windows, model versus hold RMSE is 0.041994 versus 0.023040 rad; for lower-motion windows it is 0.100089 versus 0.047816 rad. The motion threshold is the training median RMS arm displacement over a history window.

Limits of interpretation

These results do not identify a physical control fault or justify changing joint conventions. They indicate that a small model can overfit this recording and that an aggregate score can hide concentrated errors. Since task outcomes are unknown, no success percentage, expert-quality rate, or simulator score is derived from these metrics. A visual ablation and independent recordings would be required to isolate the contribution of RGB features; the ridge result alone does not establish a perception improvement.

Runtime contracts and engineering evidence

Offline inference and streaming

NumPy-only inference validates weights, state-history dimensions, normalization scales and finite outputs. The streaming runner consumes local JSONL observations with an explicit state-order and unit contract, monotonically checked timestamps and a 10 Hz history. It resets history on discontinuities or invalid observations and requires a fresh full history before predicting. Recorded receive timestamps are used for stale-input checks; none are presented as independently calibrated real clocks.

The output is explicitly diagnostic and non-executable: `robot_execution_allowed` is false and `task_success` remains null. The FR3 and gripper adapters retain their no-motion guards. The runner neither imports a motion client nor contacts a robot. Deployment requires a separately validated action mapping and closed-loop execution implementation.

Container and automated checks

The accepted 0.4.1 CPU image was reused to build the 0.4.2-smoke offline inference image without dependency downloads. The prior completed container passed 53 tests, offline inference and preview/geometry checks; preflight intentionally returned exit code 2 for missing deployment capabilities. The current source adds streaming, visual-input and archive-safety tests; local combined validation passed 74 tests at report preparation. The final candidate build log should accompany the final source snapshot.

Host CUDA availability is distinguished from container GPU support. The existing Docker daemon did not register an NVIDIA GPU runtime, and `--gpus` all failed. Therefore GPU fitting ran on the existing host environment; no container-GPU success is claimed. No driver upgrade or system reconfiguration was needed for these diagnostics.

Source package and private artifacts

The package builder uses an explicit allowlist, rejects unsafe archive paths and symlinks, verifies payload hashes and refuses existing outputs. Dataset files, recorded samples, logs, learned weights, environment directories and credentials are excluded from the source review ZIP. This package is for review; it does not establish official acceptance. Supplementary private run artifacts should be shared only after the team approves their disclosure.

Readiness, next steps and references

Current readiness

Data readability, numerical consistency, runtime importability, model fitting and offline inference have evidence. Autonomous fixture/cable perception, grasp and retention feedback, verified robot action mapping, calibrated task coordinates, recovery control, observation-based completion, and organizer-host integration remain incomplete. Simulation task success and real-robot task success are untested. The work therefore supports a technical-progress report, not a finished runnable-policy submission.

Prioritized completion plan

First, resolve organizer interfaces, camera calibration, task ordering, gripper semantics and stop/timeout behavior. Second, review independent attempts and outcome labels, especially Y checkpoints and plugging, and establish a genuinely independent test recording. Third, evaluate scene-conditioned actions against hold and simple residual baselines with per-stage and per-joint errors. Fourth, validate closed-loop execution and completion criteria in a suitable simulator and then through organizer-run real testing. Training duration alone cannot replace these dependencies.

Only bounded diagnostic runs have been executed. Further formal training, public upload and official Issue creation remain subject to team review and the previously agreed budget/disclosure constraints. No private dataset, robot endpoint or authentication information is included in this report.

References and provenance

[1] EBiM-Benchmark, submissions repository README. <https://github.com/EBiM-Benchmark/submissions> . Accessed 10 September 2026. It describes the Technical Report alternative and registration verification; the team must resolve the report-form availability before filing.

[2] EBiM Competition, schedule and Task 1 description. <https://ebim-benchmark.github.io/competition.html> . Accessed 10 September 2026. Full-autonomy requirements apply to runnable task evaluation; the published general Phase II deadline is 12 September 2026 AoE, subject to any team-specific organizer instructions.

[3] EBiM-Benchmark, current Issue forms. https://github.com/EBiM-Benchmark/submissions/tree/main/.github/ISSUE_TEMPLATE . Accessed 10 September 2026. Current runnable-policy form and any team-specific deadline must be checked against advancement correspondence.

[4] GG boy Team internal evidence, 8-10 September 2026: read-only dataset transfer/replay acceptance; saved training configurations, split indices, predictions, exported weights, independent verification and container logs. Private raw data are not redistributed.

Source provenance: the original real_robot migration layer derives from BigCatwanzi20071202/ebim-task1-mujoco-submission at local revision b717b77783415e3e0e27b1044dcebd30bda6135b. The original LICENSE and UPSTREAM_NOTICE remain in the review package. No simulator privileged-state technique is claimed as a real-robot perception result.